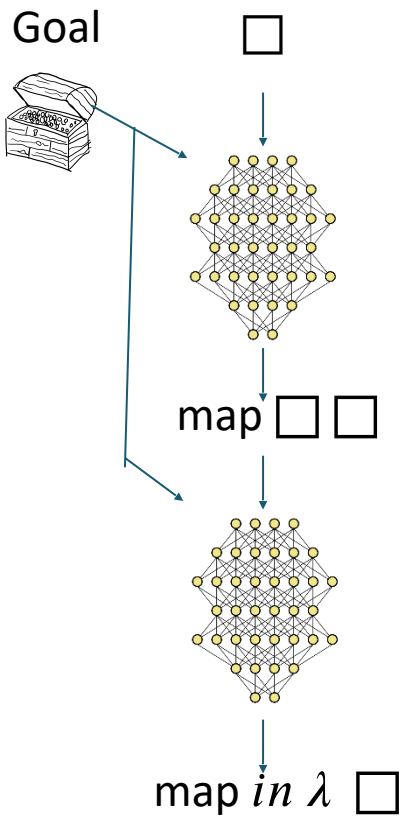
Lecture 24
Neurosymbolic Synthesis

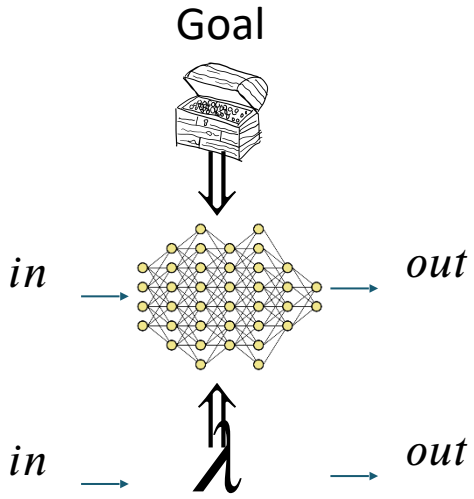
Armando Solar-Lezama with slides from Swarat Chaudhuri

Neurosymbolic program synthesis

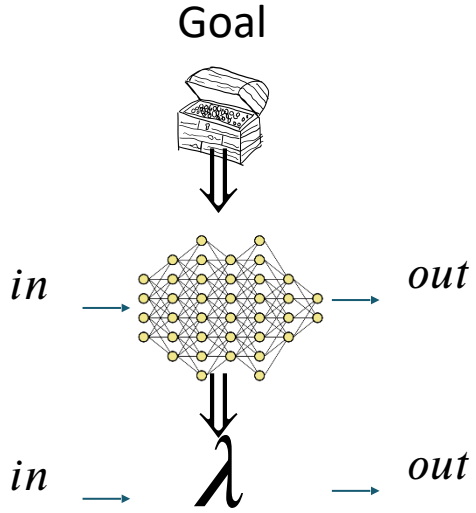
Neural Guided Search



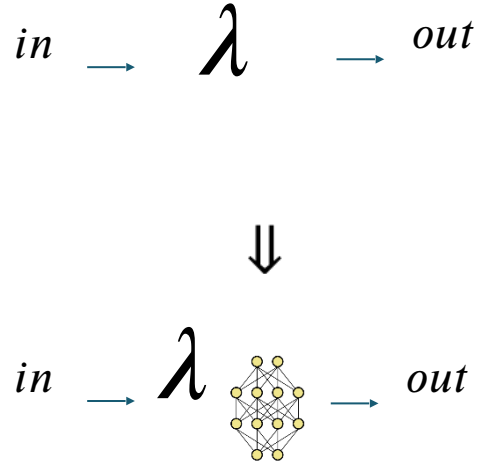
Symbolic Guided DL



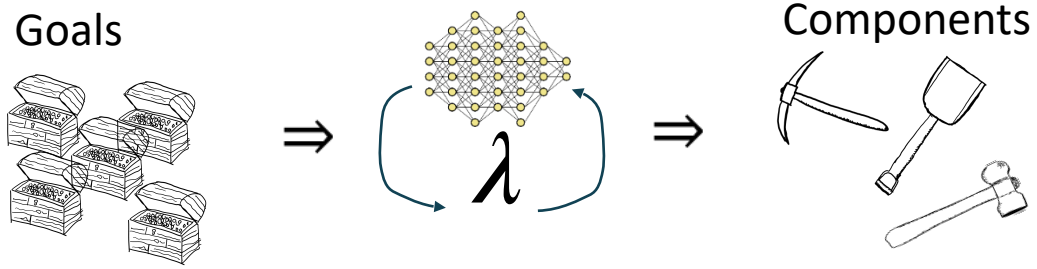
Distillation



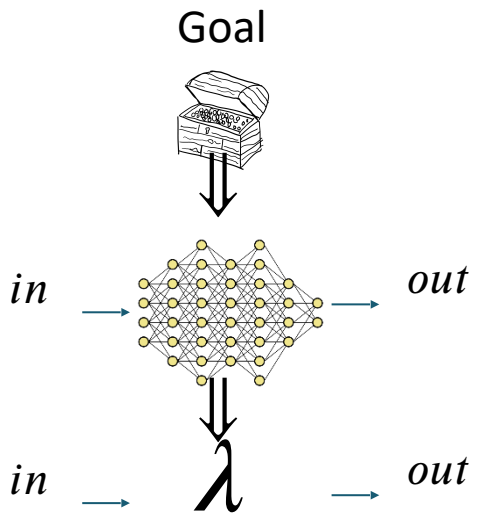
Relaxation



Component Discovery



Distillation



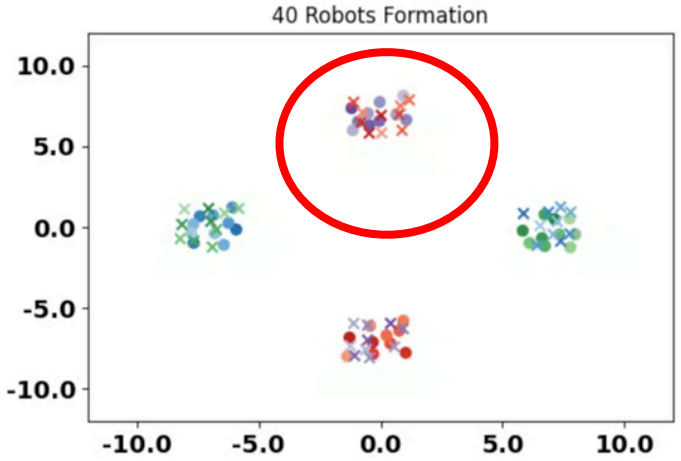
Use the neural network as a starting point for program synthesis

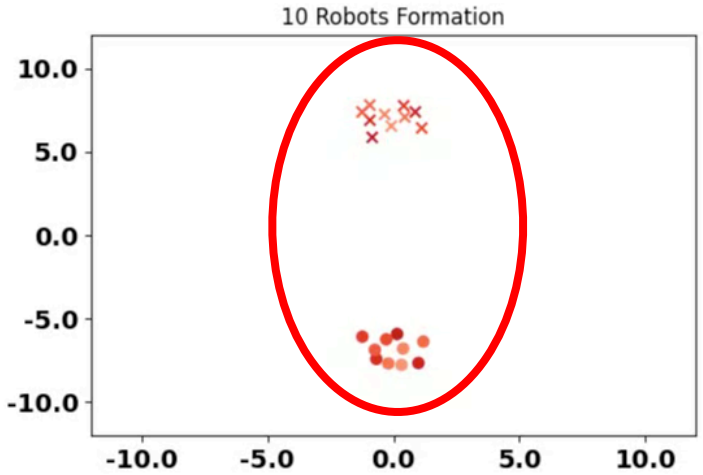
Replace neural components with symbolic ones

- To improve interpretability and analyzability
- To better generalize out of distribution
- To ensure more predictable behavior

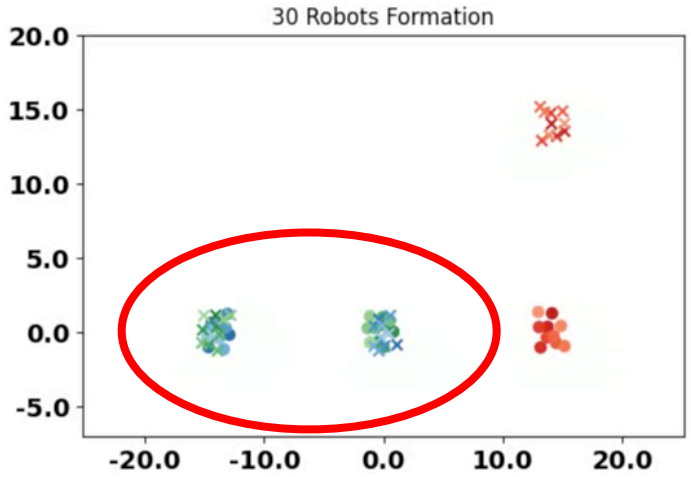
Multi-Agent Systems

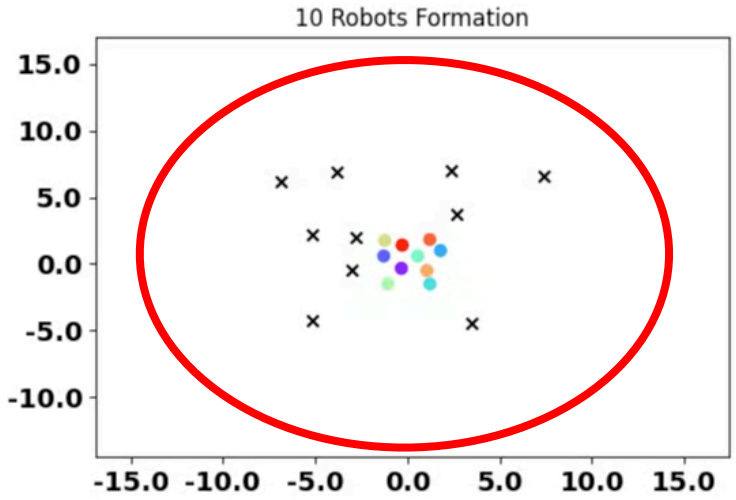
Random Cross





Random Grid

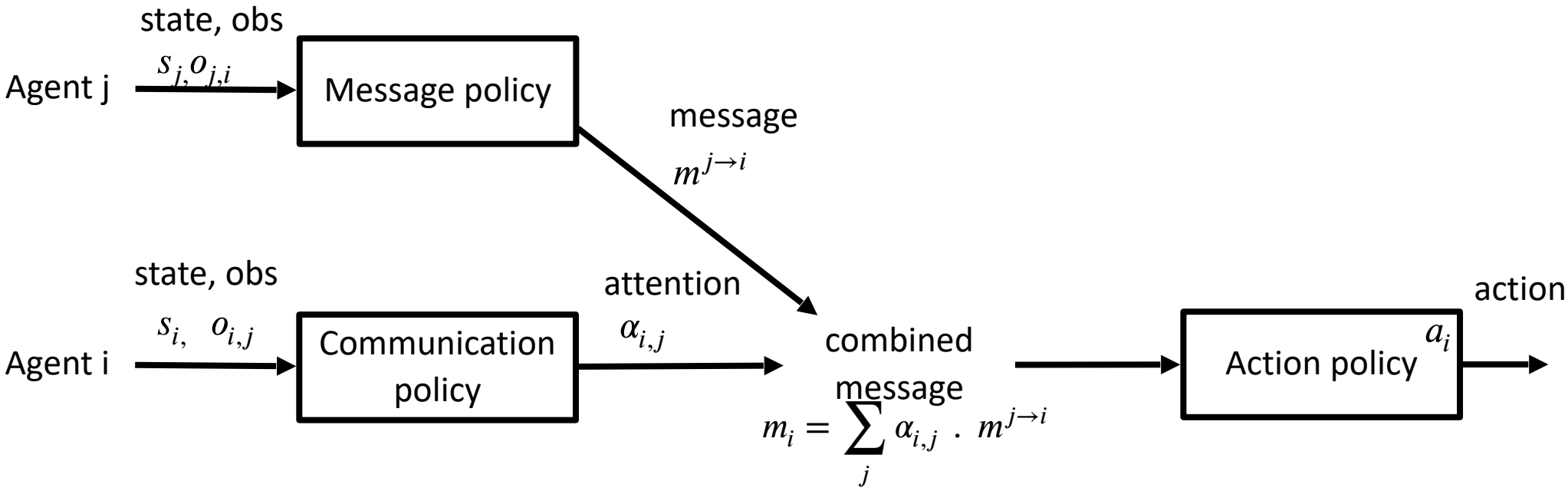




Unlabeled goals

Jeevana Priya Inala, Yichen Yang, James Paulos, Yewen Pu, Osbert Bastani, Vijay Kumar, Martin Rinard, Armando Solar-Lezama

Decentralized policy



Communication constraint: minimize

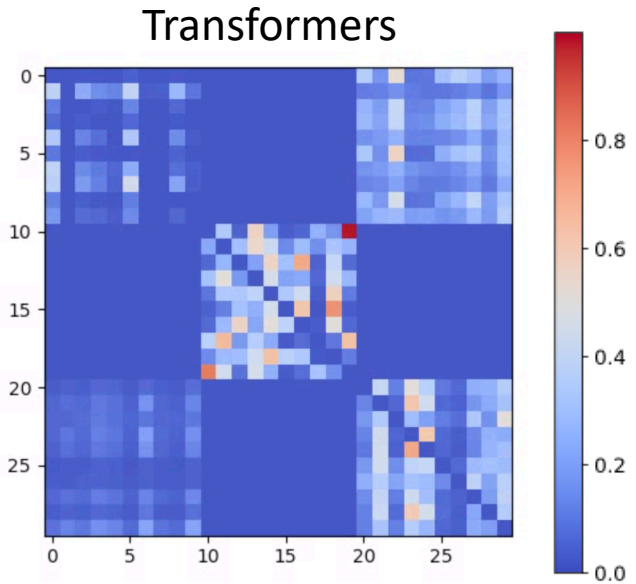
$$\max_i \sum_j \mathbf{1}[\alpha_{i,j} > 0] + \max_i \sum_j \mathbf{1}[\alpha_{j,i} > 0]$$

Maximum incoming degree

Maximum outgoing degree

Alternate Approaches

Transformer



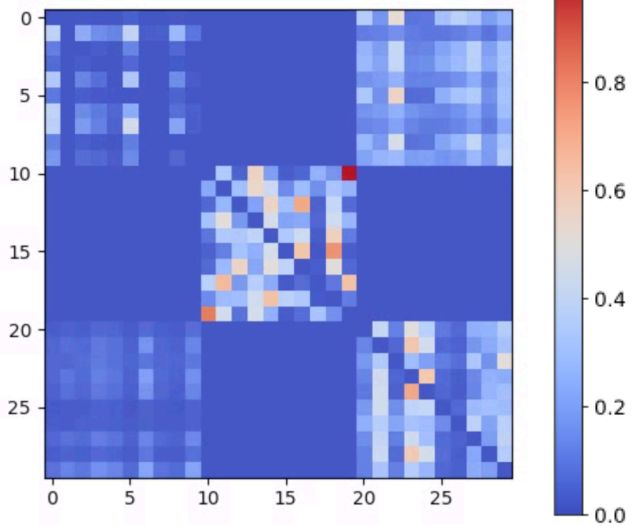
Das et. al., *ICML 2019*

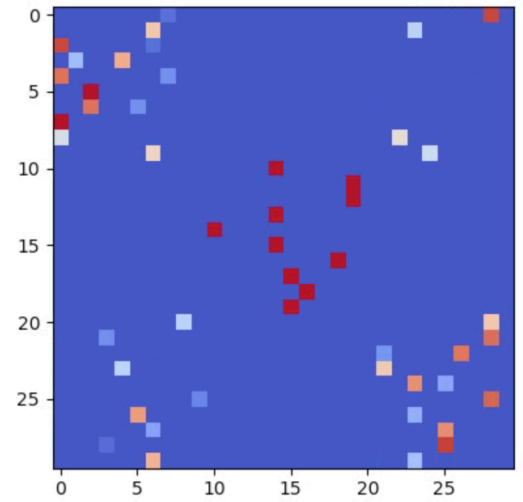
Alternate Approaches

Transformer

Our approach: program from a custom DSL

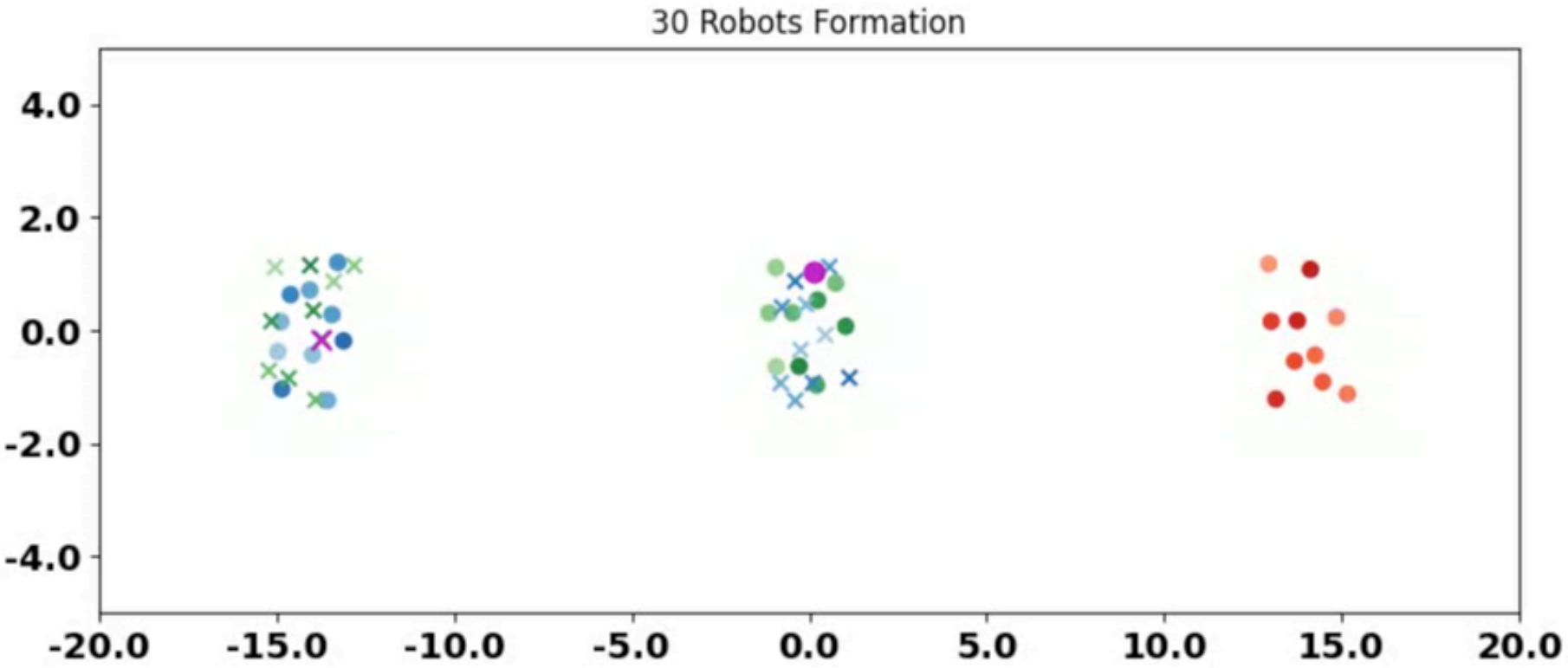
Transformers





Program

Programmatic communication rules



Deciding agent

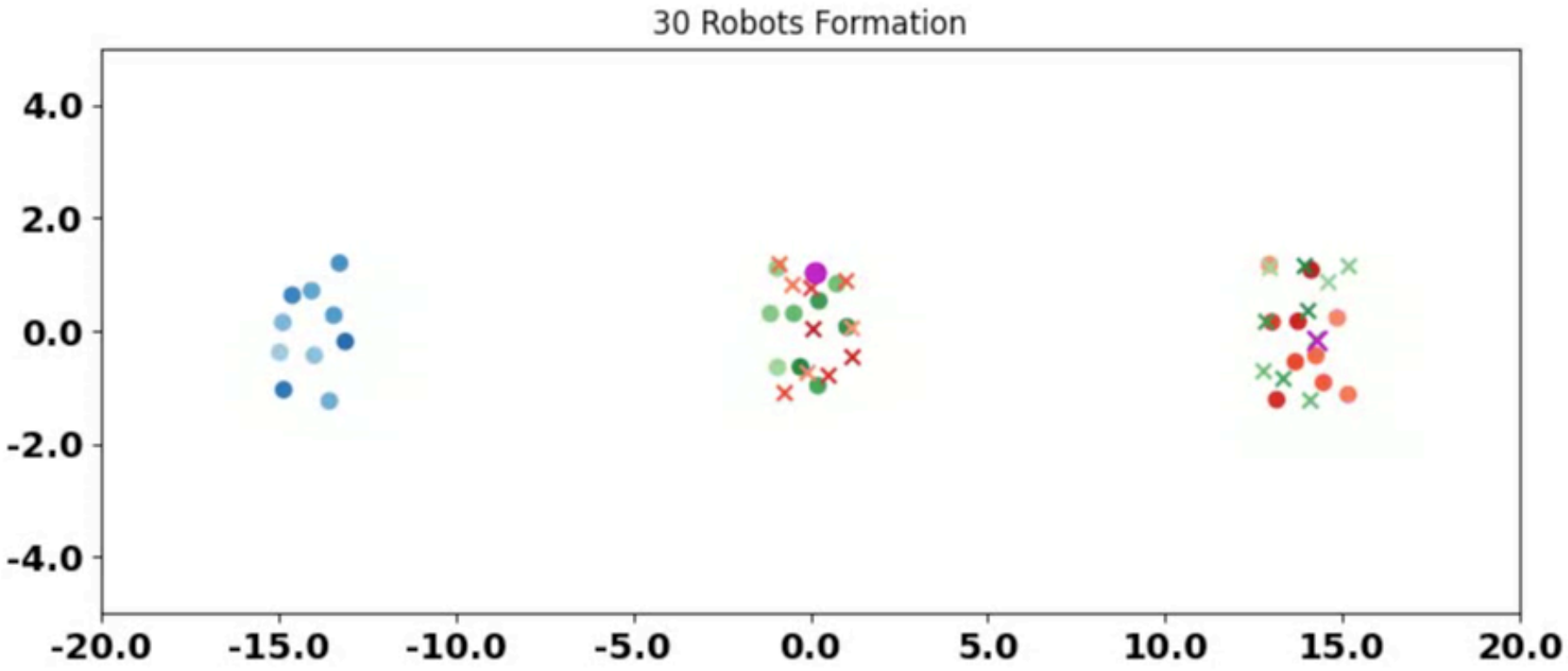
Deciding agent's goal

A → B: A is sending message to B

Rule1 :: random (filter (agents in ■)) θ

The type of rule and the parameters θ need to be learned

Programmatic communication rules



Deciding agent

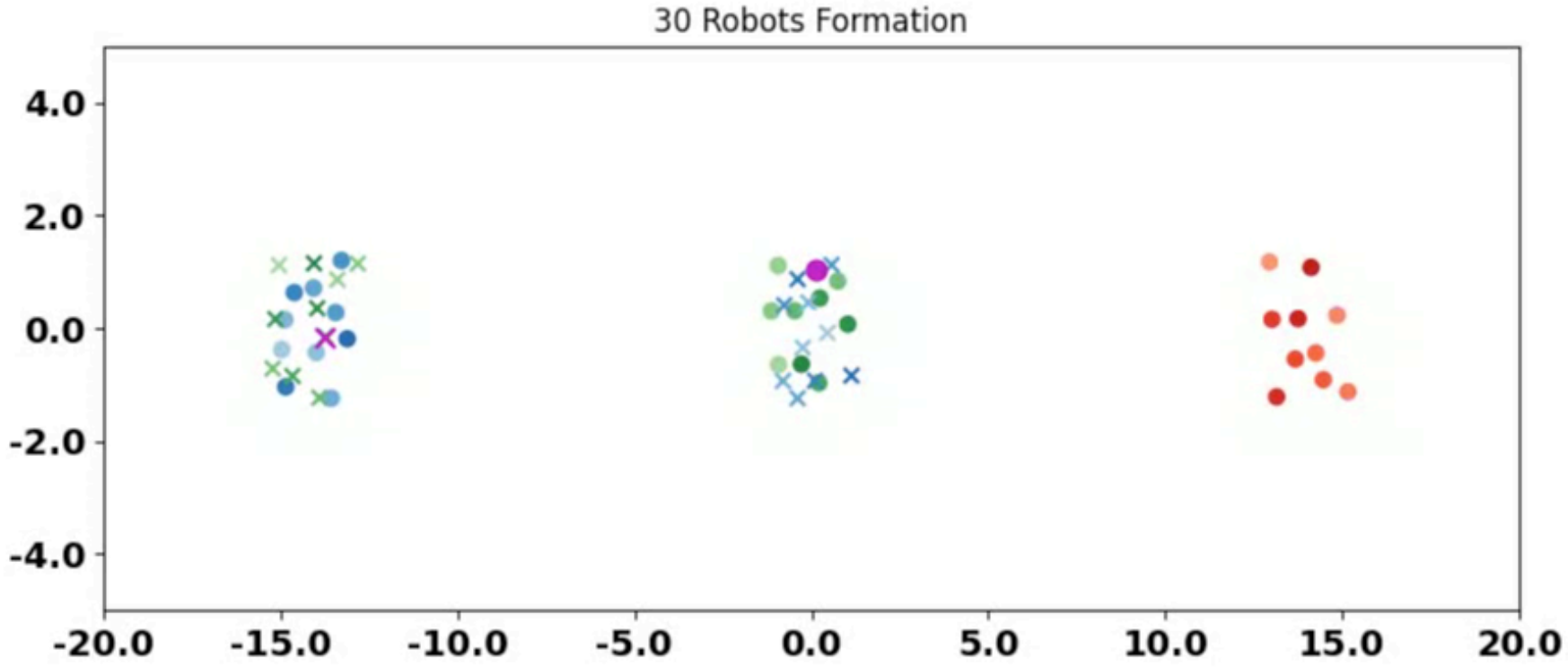
Deciding agent's goal

A → B: A is sending message to B

Rule1 :: random (filter (agents in  **))**

The type of rule and the parameters θ need to be learned

Programmatic communication rules



Deciding agent

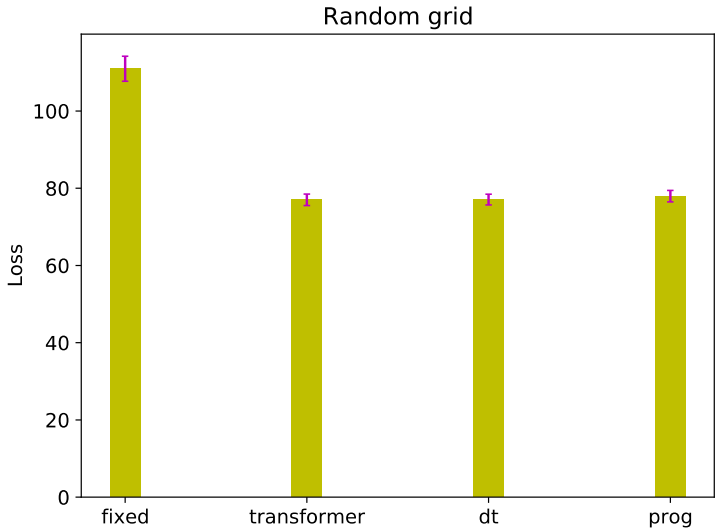
Deciding agent's goal

A → B: A is sending message to B

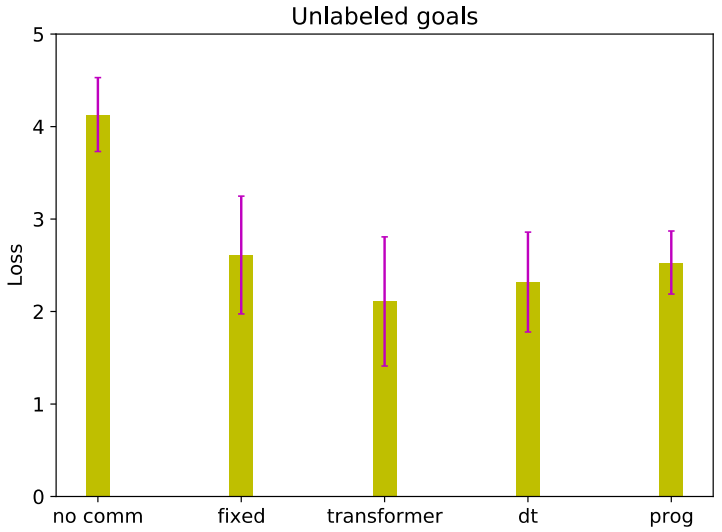
Rule2 :: argmax (map (f_θ, filter (agents in θ))

The type of rule and the parameters θ need to be learned

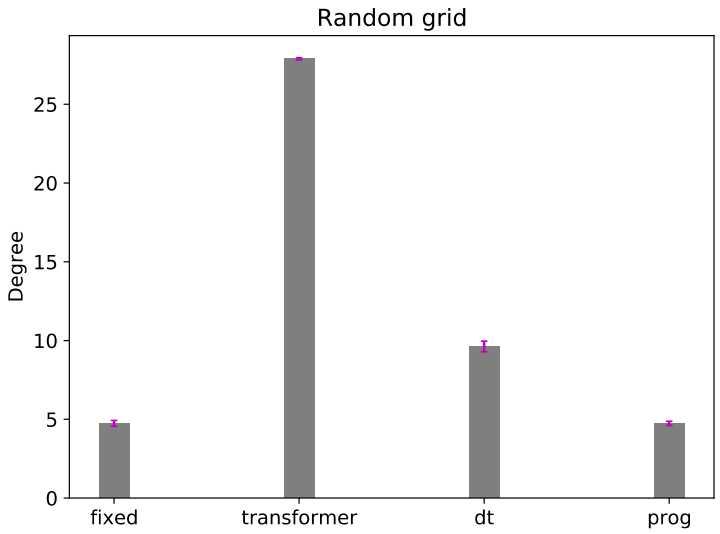
Results: Good performance

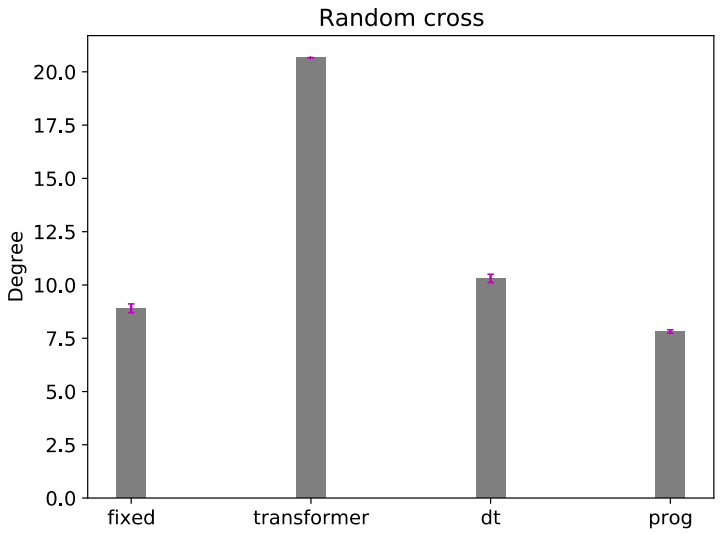


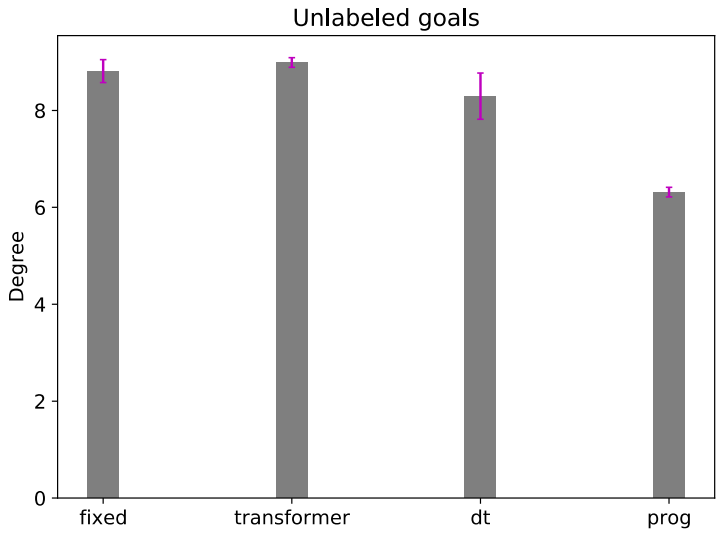




Results: Reduces communication degree





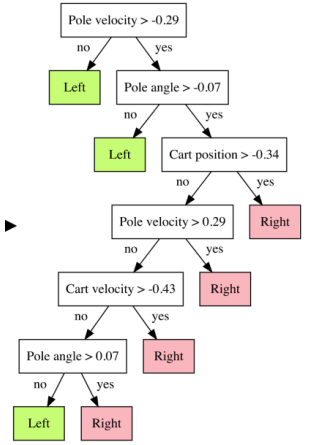


Other similar work

$$\textbf{if } (\textbf{fold}(+, h_0) - \textbf{peek}(h_3, -1)) > 0$$
$$\quad \textbf{then } 0$$
$$\quad \textbf{else } 1$$

Verma et. al. ICML 2018, NeurIPS 2019

Distil symbolic controllers from a learned neural policy



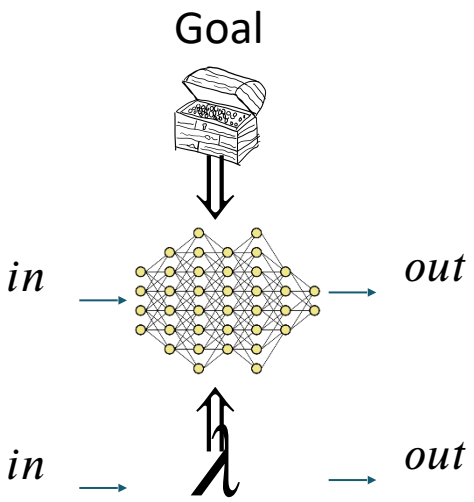
Bastani et. al. NeurIPS 2018

Distil a decision tree from a neural policy and verify against a specification

Combinations

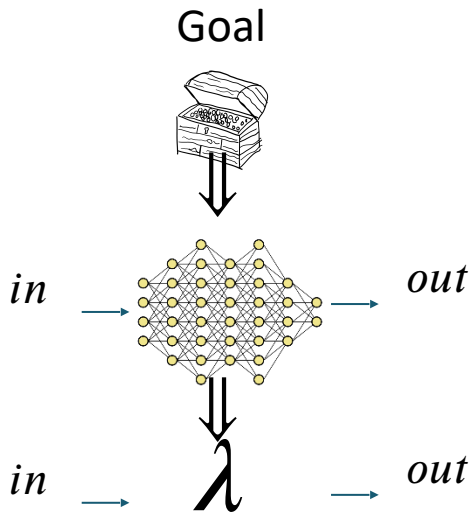
Combinations

Symbolic Guided DL

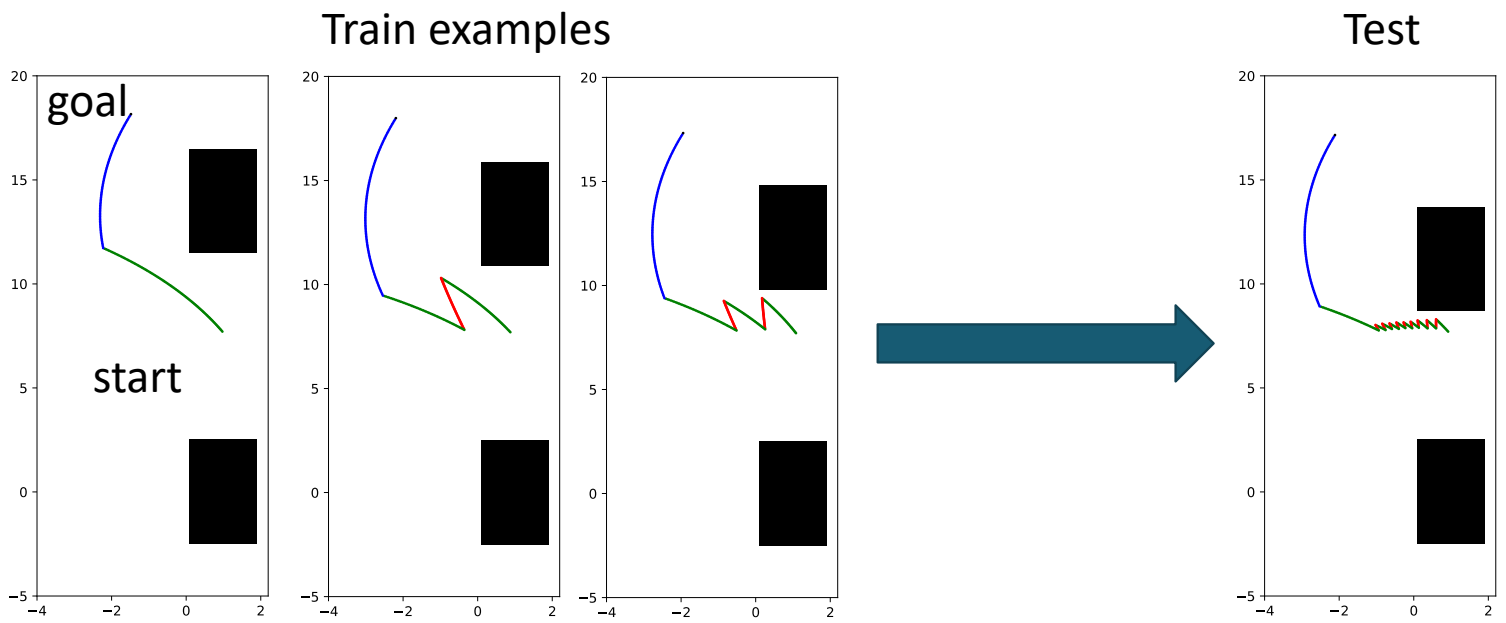


+

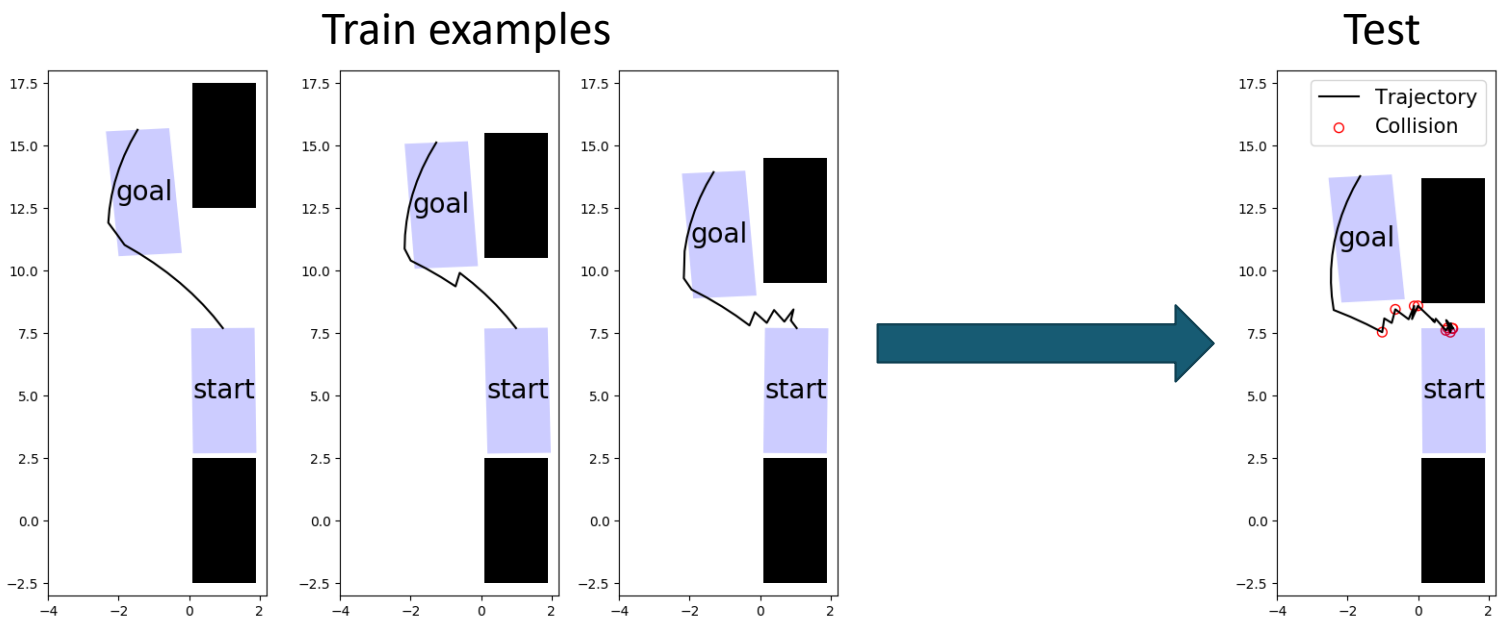
Distillation



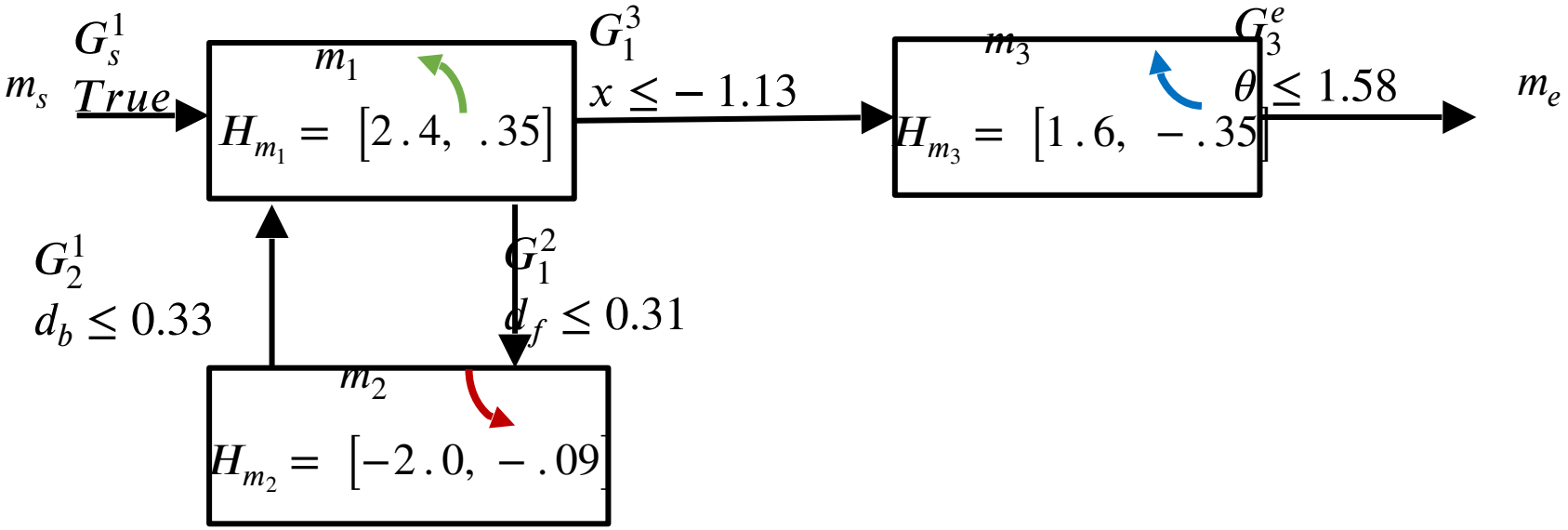
Inductive Generalization

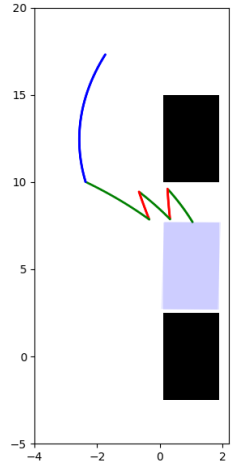


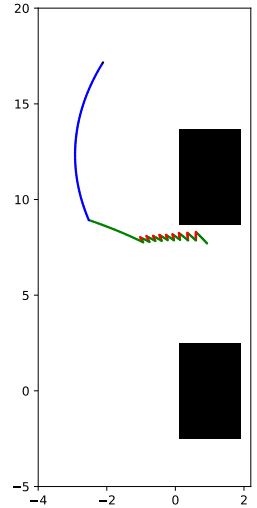
Inductive Generalization – NN fails



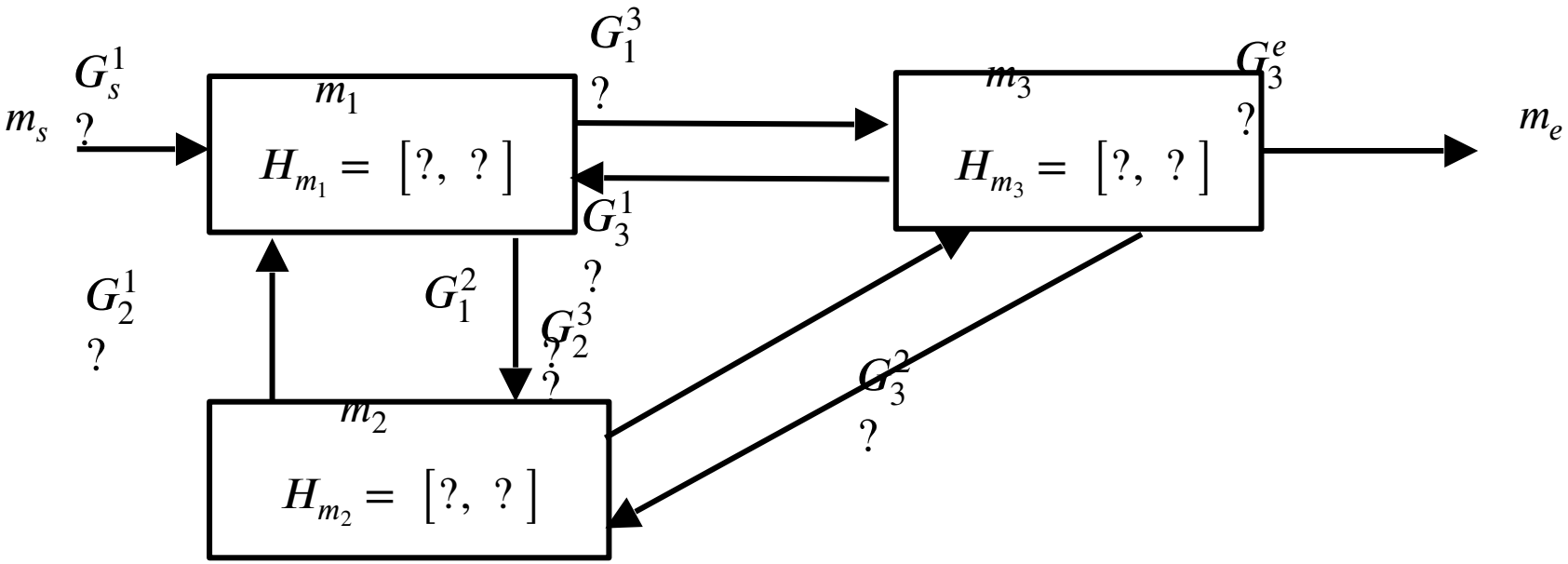
Programmatic Policies based on State Machines







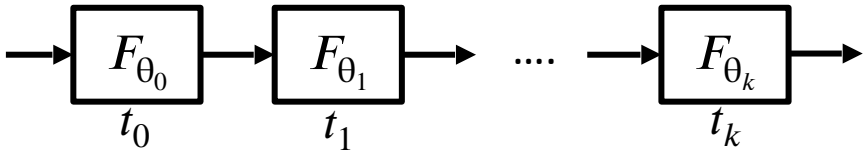
How to learn programs?



Direct numerical optimization does not work

Over-parameterization

Loop-free policy (τ)



Straight-chain of simple policies

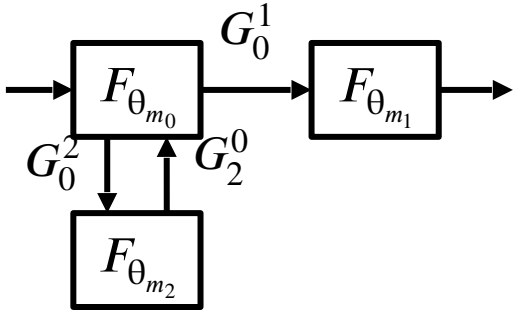
Transitions based on time

Different policies for different initial states

Fully continuous, easy to learn

Not generalizable

State-machine policy (π)



Simple policies connected as a graph

Transitions based on a function on state

Same policy for all initial states

Discrete + continuous, hard to learn

Generalizable

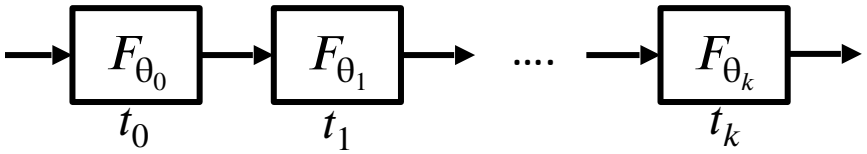
Teacher-Student algorithm

Teacher

Learns loop-free policies that maximize reward

$$x_0^k \sim X_0 \qquad \tau_k = \operatorname{argmax}_\tau R(\tau,\ x_0^k)$$

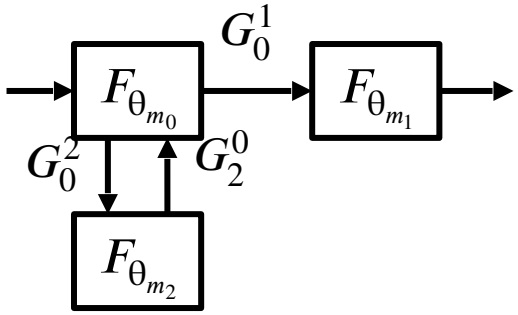
trajectory



Student

Learns state-machine policy that imitates loop-free policies

$$\pi^* = \operatorname{argmax}_\pi \sum_k \log p(\tau_k \mid \pi, x_0^k)$$

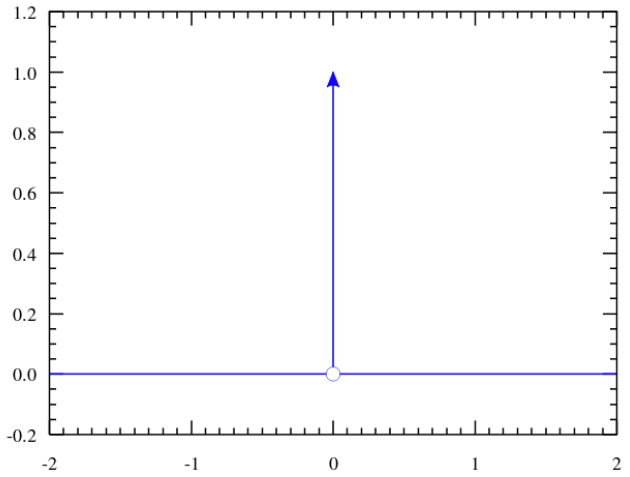


Over-parameterization

State machine policy to loop-free policy



Issue: $p(\tau \mid \pi, x_0)$ **is degenerate**



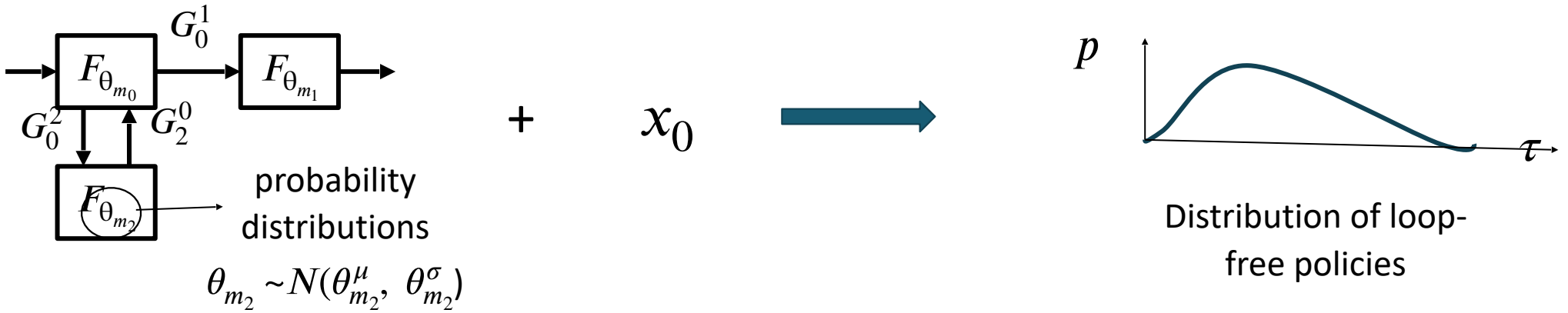
Over-parameterization

State machine policy to loop-free policy



Issue: $p(\tau \mid \pi, x_0)$ **is degenerate**

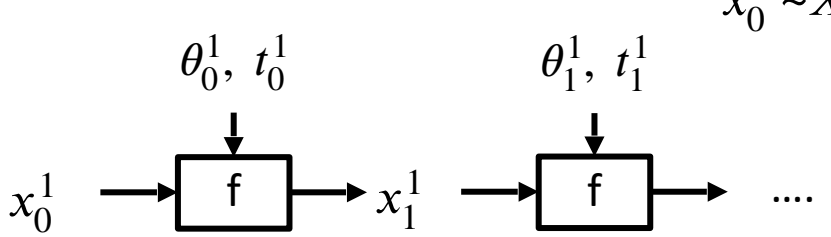
Solution: Make state machine policy probabilistic

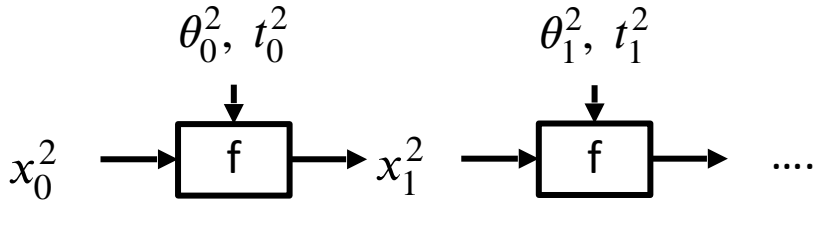


Algorithm

Teacher: Maximizes reward on an over-parameterized representation of policies (loop-free policies)

$$x_0^k \sim X_0 \qquad \tau_k = \underset{\tau}{\operatorname{argmax}} R(\tau,\ x_0^k)$$







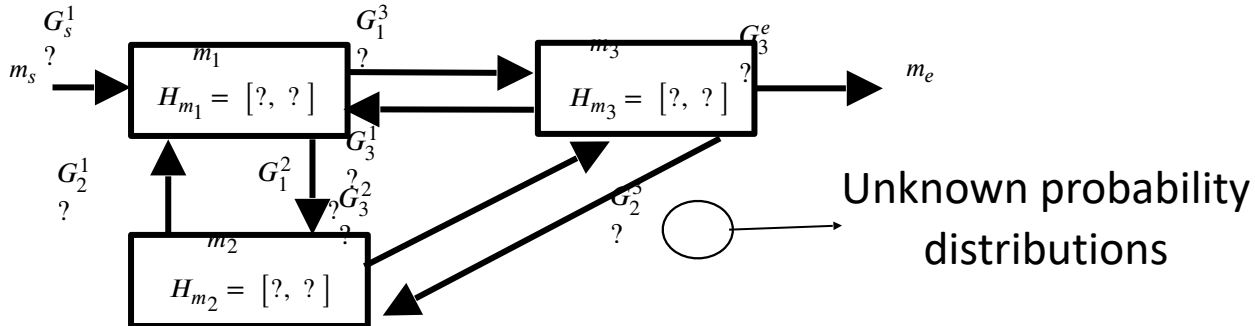
τ_1



τ_2

Student: Imitate loop-free policies

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \sum_k \log p(\tau_k \mid \pi, x_0^k)$$



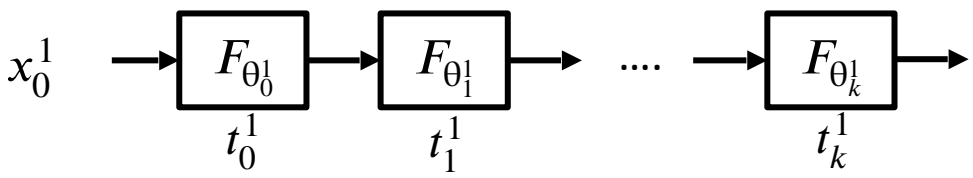
π: Prob program policy

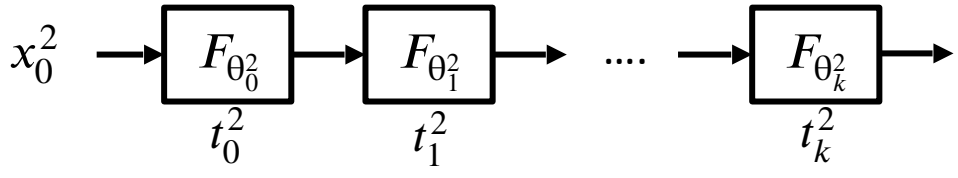
τ: Loop-free policy

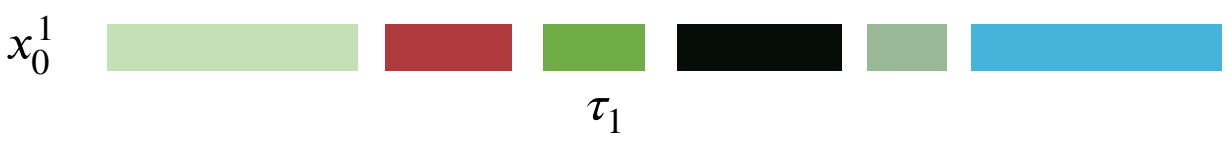
R: Reward function

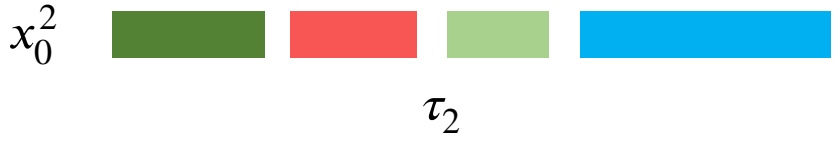
Student - Imitation Learning

Loop-free policies from the teacher

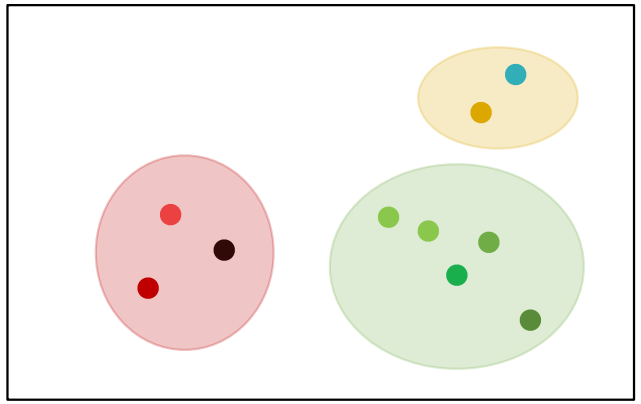


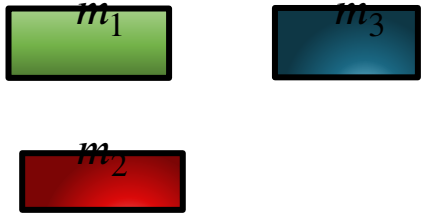






Step 1: Learn actions for the different modes



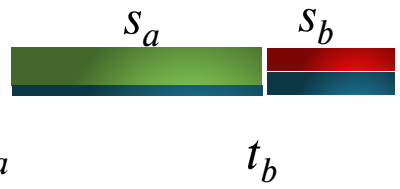


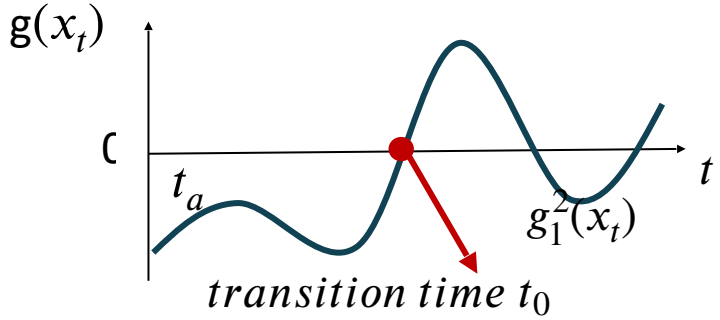


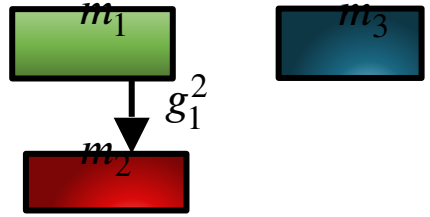


Student – Imitation Learning

Step 2: Learn switching conditions

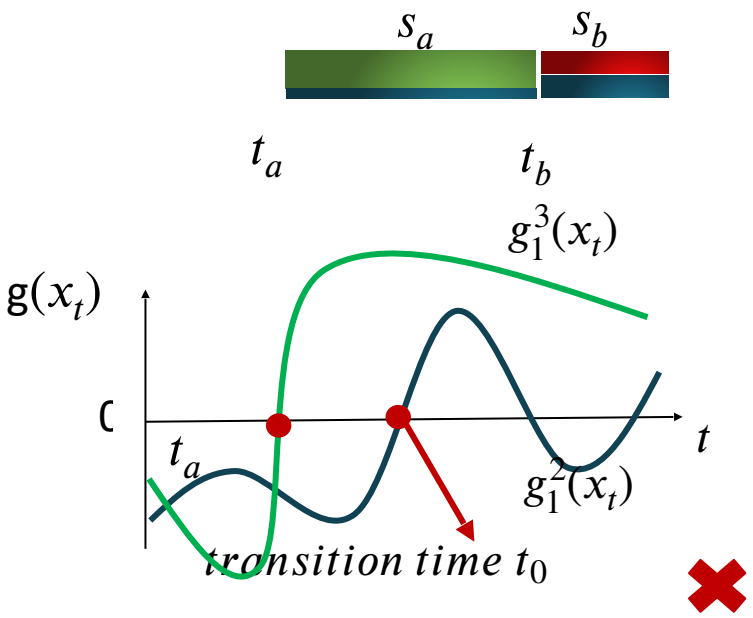


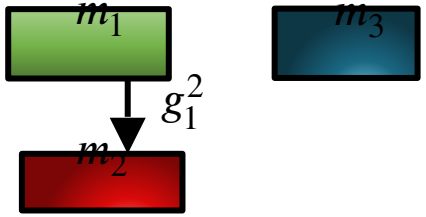




Student – Imitation Learning

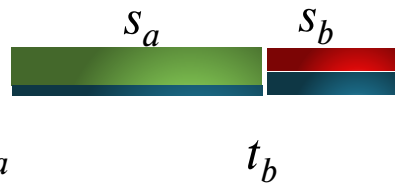
Step 2: Learn switching conditions

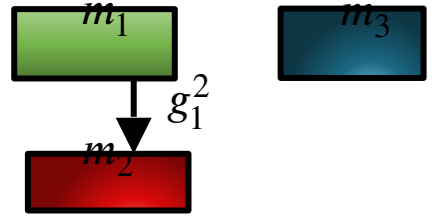


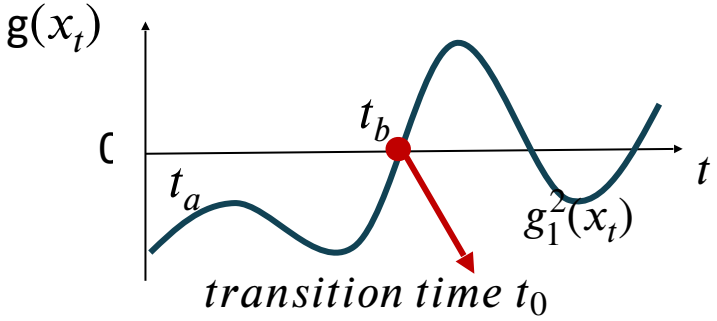


Student – Imitation Learning

Step 2: Learn switching conditions



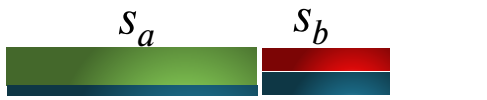


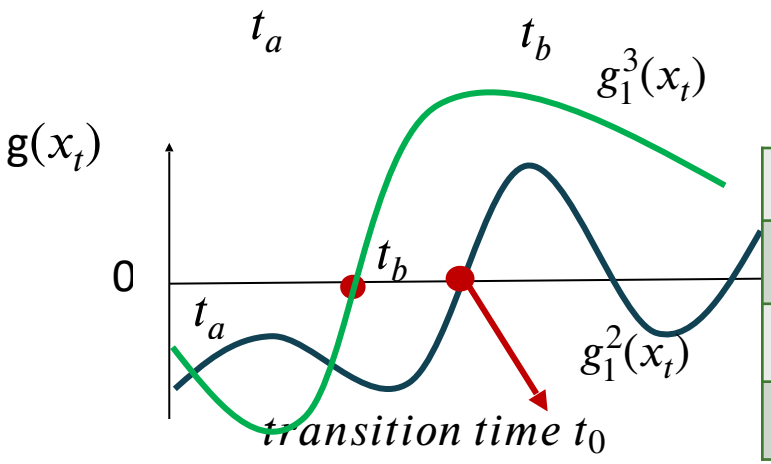


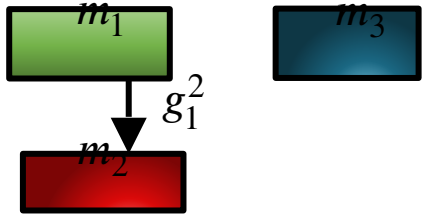
Scenario	Probability of scenario	Objective
		log
	)	log
	(1 -	0

Student – Imitation Learning

Step 2: Learn switching conditions





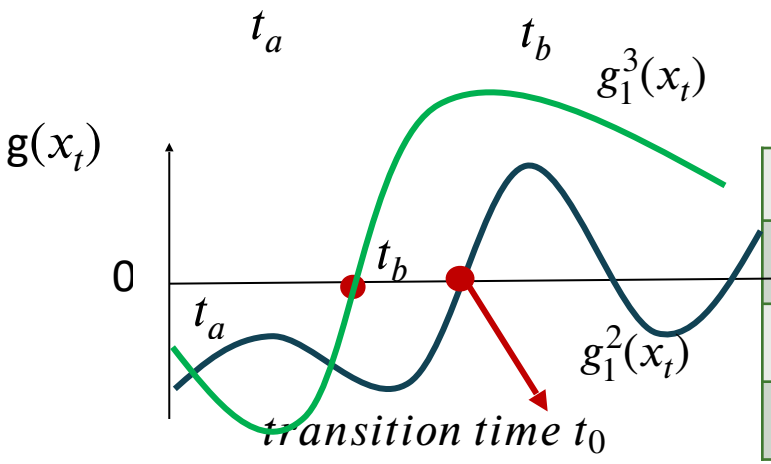


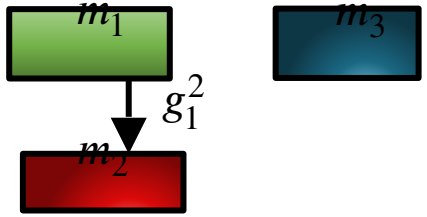
Scenario	Probability of scenario	Objective
		log
	)	log
	(1 -	0

Student – Imitation Learning

Step 2: Learn switching conditions





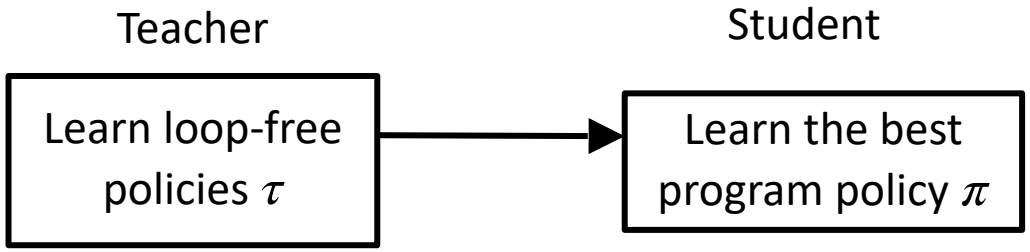


Scenario	Probability of scenario	Objective
		log
	)	log
	(1 -	0

Full objective

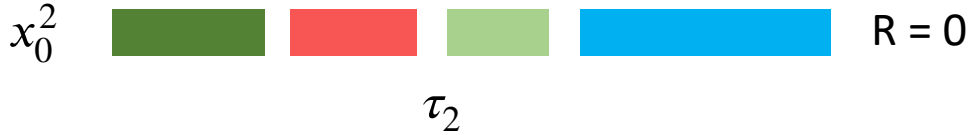
$$\underset{g_1^2}{\operatorname{argmax}} \sum_{s_a,\, s_b} p(s_a \text{ is } m_1)\, p(s_b \text{ is } m_2) \log P(t_0 = t_b \mid g_1^2,\, s_a) \;+\; p(s_a \text{ is } m_1)\left(1 - p(s_b \text{ is } m_2)\right) \log P(t_0 > t_b \mid g_1^2,\, s_a)$$

Teacher-student algorithm



Loop-free policies from the teacher



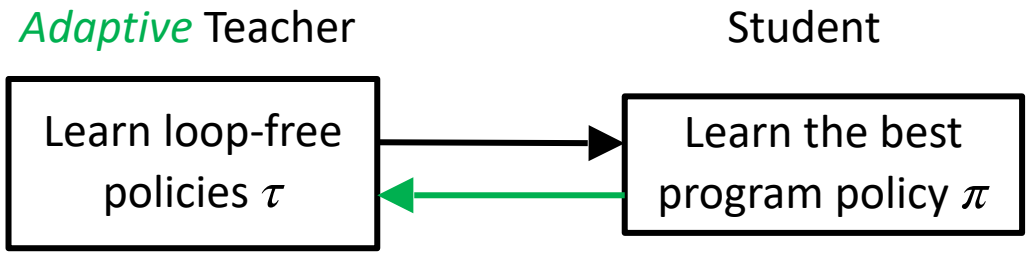


Best rollouts from the learned state machine





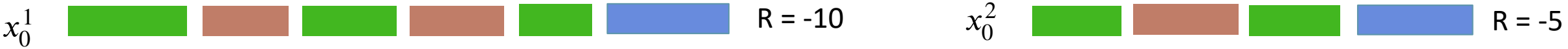
Adaptive Teaching



Loop-free policies from the teacher



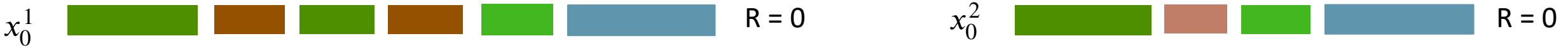
Best rollouts from the learned state machine



Adaptive Teacher: Maximizes reward on loop-free policies as well as imitates student

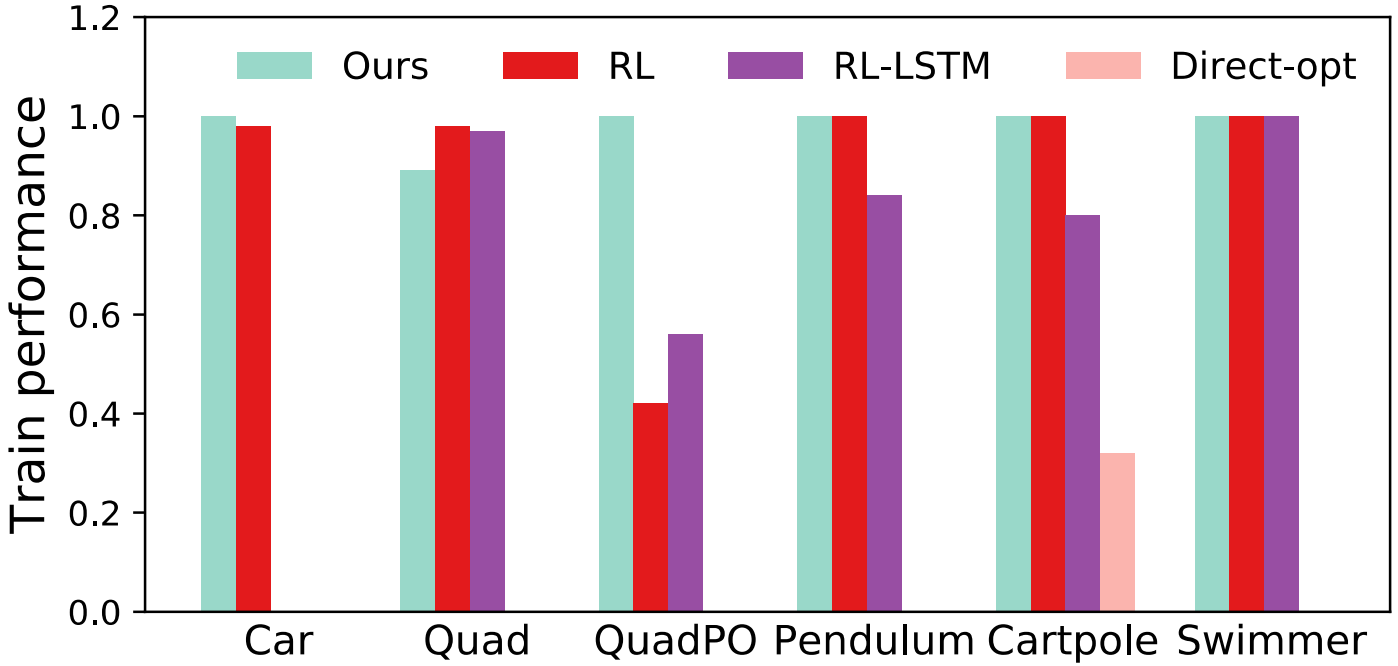
$$x_0^k \sim X_0 \qquad \tau_k = \operatorname{argmax}_\tau R(\tau,\ x_0^k) + \lambda \log p(\tau \mid \pi,\ x_0^k)$$

Loop-free policies from the teacher (iteration 2)



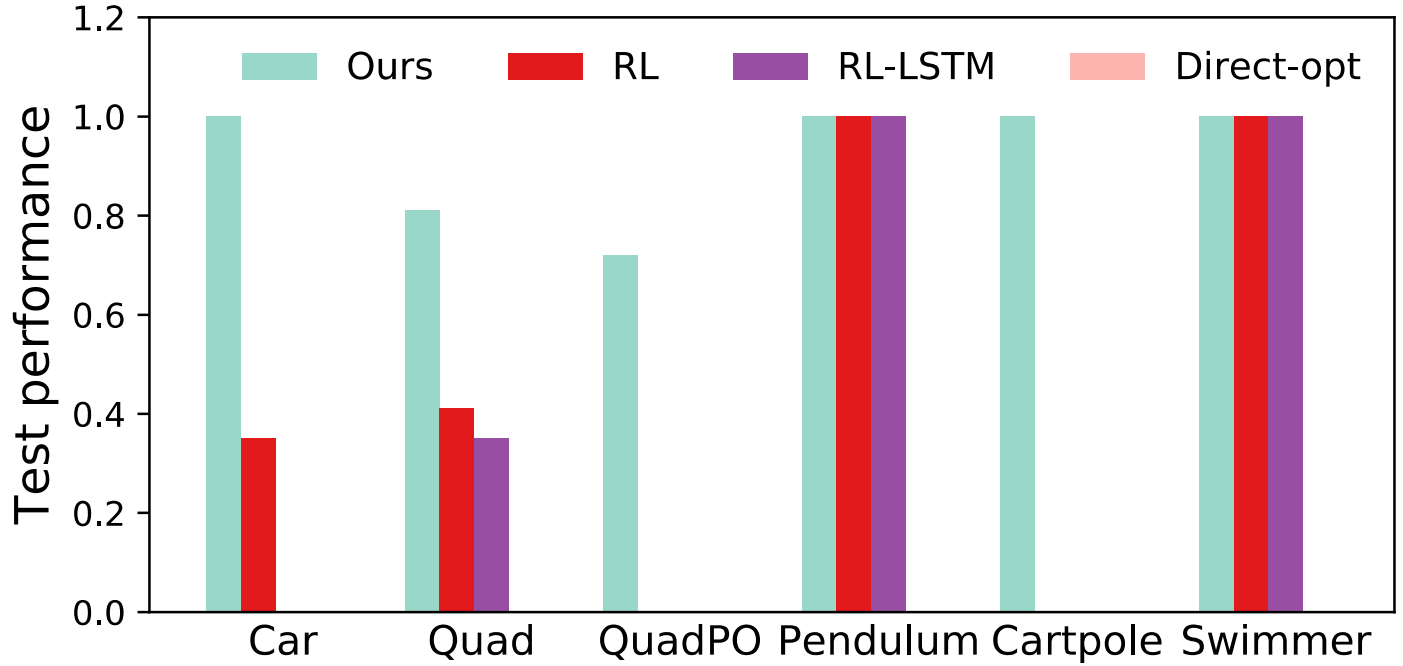
Results: Language is Expressive

On the **training set**, both our approach and reinforcement learning approaches **perform equally.**

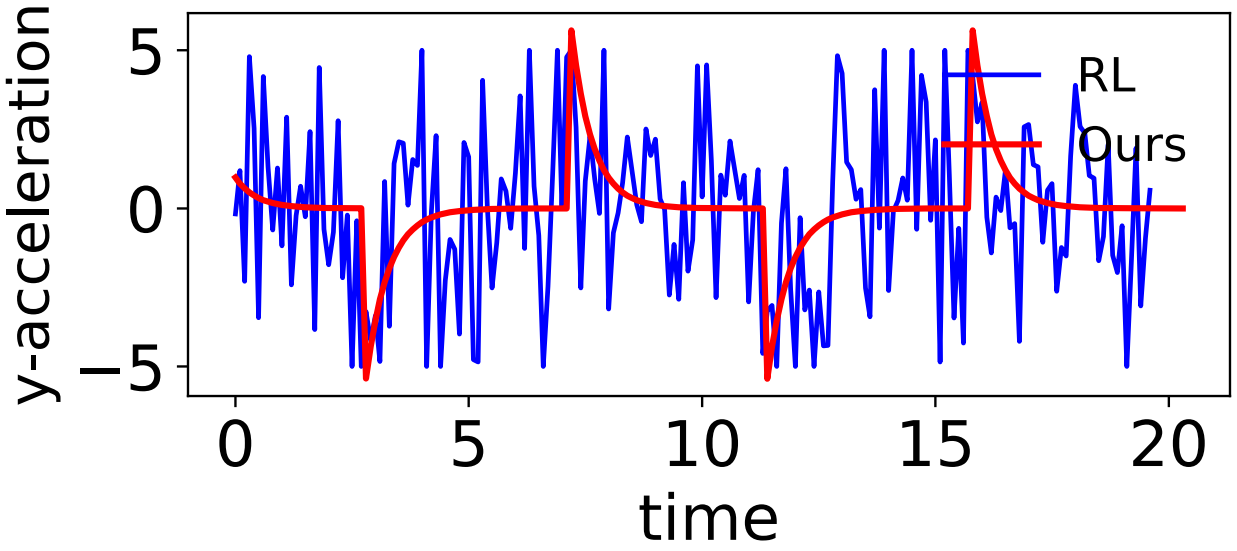


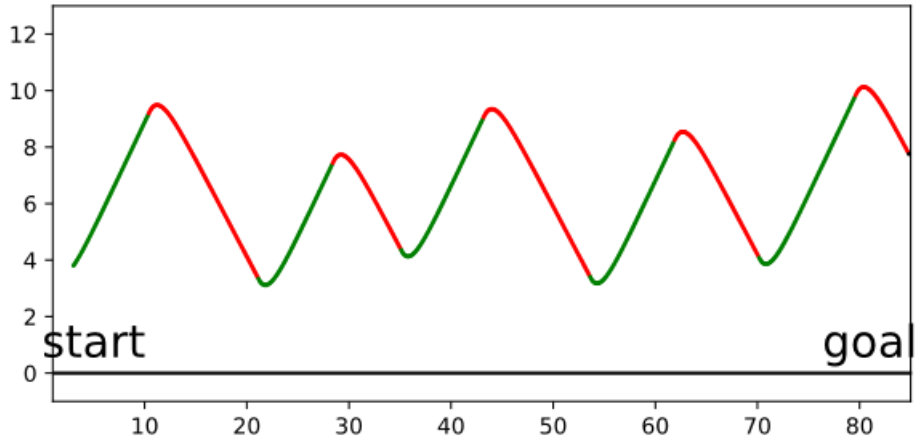
Results: Inductive Generalization

On the **test set**, **our approach shows better generalization**.



Results: Smoother actions

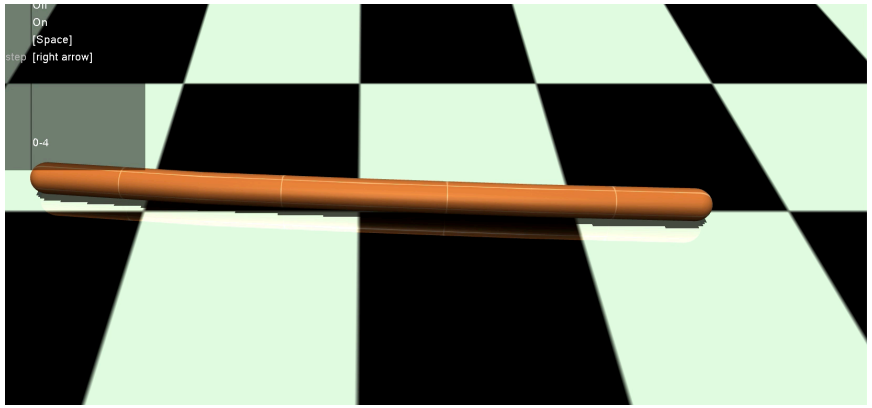




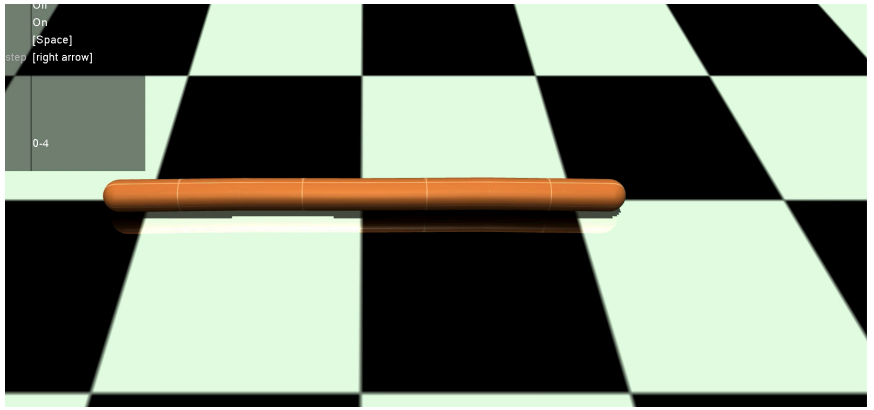
Trajectory of a quadcopter navigating an obstacle course

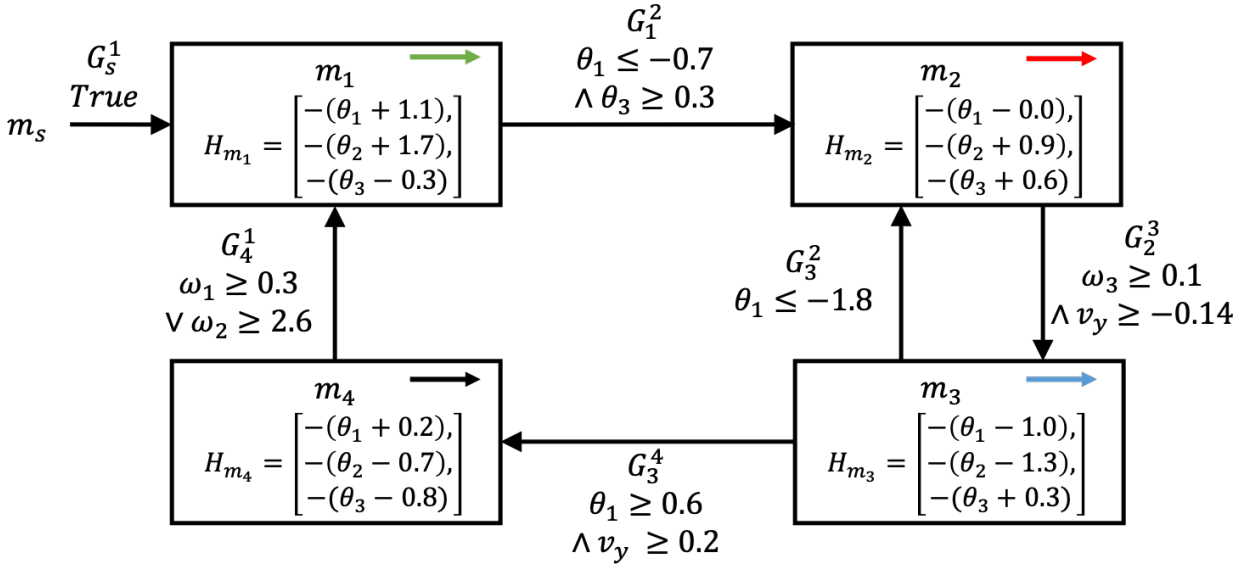
Results: Complex policies

Train: segmets len = 1



Test: segments len = 0.75





Results: Interpretability

